ShapeCodeBench: A Renewable Benchmark for Perception-to-Program Reconstruction of Synthetic Shape Scenes*

Shivam Kumar
Independent Researcher
shivam@shivamk3r.com
https://shivamk3r.com/

May 11, 2026

Abstract

We introduce ShapeCodeBench, a synthetic benchmark for *perception-to-program reconstruction*: given a rendered raster, a model must emit an executable drawing program that a deterministic evaluator re-renders and compares. The DSL has four primitives on a 512×512 black-on-white canvas, but every instance is generated from a seeded RNG, so fresh held-out sets can be minted to mitigate benchmark contamination. Because both instance generation and scoring are automatic, the same loop can refresh evaluations quickly without per-instance human annotation or manual judging. We release a frozen split, eval_v1 (150 samples, 50 per difficulty tier), scored by exact match, pixel accuracy, and foreground IoU alongside parse and execution success. Evaluating four reasoning-effort configurations of two frontier multimodal models – Claude Opus 4.7 (1M context) at `high` and `max` effort, and GPT-5.5 at `medium` and `extra_high` reasoning effort – against an empty-program floor and a classical-CV heuristic baseline exposes a tier-dependent crossover: the heuristic leads easy-tier exact match (0.26 vs. at most 0.08 for any multimodal configuration) by individuating separated connected components, but collapses on medium and hard scenes as overlapping shapes fuse; the strongest multimodal model by foreground IoU (GPT-5.5 at `extra_high` effort) retains most of the spatial structure and leads foreground IoU on every tier (up to 0.87), yet misses exact match by small parameter errors, while Claude Opus 4.7 (1M) trails the heuristic on foreground IoU at both effort levels. Best overall exact match is 0.087 (heuristic) and 0.027 among multimodal models, so ShapeCodeBench is far from saturated. Benchmark code, frozen dataset, and full run artifacts are released to support independent replication and extension.

1 Introduction

Modern multimodal models are increasingly evaluated on their ability to turn images into code. Work on screenshot-to-HTML [2, 16], structure extraction from webpages, LaTeX, and music scores [14], and symbolic vector generation [10, 19] all ask the same underlying question in different clothes: *can a model look at a picture and produce the program that generated it?* Earlier work on visual program induction [15, 12, 9, 8, 4, 5] framed this problem as inverse graphics over a small symbolic DSL, and more recently TurtleBench [13] has revived it as a benchmark-first target for large vision-language models.

Across these lines of work, three design pressures keep recurring: (1) deterministic execution so that scoring is principled; (2) render-based scoring so that semantically equivalent programs are not

*Code, benchmark data, and figures: https://github.com/shivamk3r/shape-code-bench

penalized for textual differences [14]; and (3) controlled generation so that failure modes can be attributed to specific visual factors, in the tradition of CLEVR [6, 1]. Most existing benchmarks satisfy one or two of these, but few satisfy all three while remaining *renewable* – that is, cheap enough to regenerate when an existing split becomes contaminated. For model development, renewability also changes the feedback cycle: a researcher can generate fresh instances, run a model, and obtain objective scores without commissioning new labels or manual judgments for each refreshed example.

We present ShapeCodeBench, a perception-to-program benchmark that attempts to hit the intersection of these pressures. The task is narrow by design: the DSL has exactly four primitives (`filled_circle`, `circle`, `filled_square`, `square`) and the canvas is fixed at 512×512 grayscale with a black foreground on a white background. Every sample is generated from a seeded RNG with explicit difficulty controls on shape count, size, stroke width, overlap, and canvas clipping. The evaluator parses predictions with a safe restricted Python parser, re-renders them through the same deterministic Pillow pipeline used to produce the target, and compares rasters.

Contributions. We make the following contributions:

1. We release the ShapeCodeBench benchmark: a four-primitive drawing DSL, a safe restricted parser, a seeded scene generator with three difficulty tiers, and a render-based evaluator with five primary metrics.
2. We freeze an evaluation split, eval_v1, of 150 samples with deterministic seeds, and publish per-sample raster hashes to make the exact evaluation instances reproducible across platforms.
3. We make benchmark refresh a first-class workflow: new held-out splits can be generated from fresh seeds and scored automatically, enabling fast regression-style feedback while avoiding per-instance human annotation or manual judging. This mitigates exact-instance contamination but does not prevent models from learning the generator distribution.
4. We release a provider-agnostic runner that records prompts, model configuration, raw outputs, normalized predictions, metrics, and per-sample artifacts, making model evaluations auditable and easy to extend.
5. We report baseline results for four multimodal model configurations – Claude Opus 4.7 (1M context) at `high` and `max` effort, and GPT-5.5 at `medium` and `extra_high` reasoning effort – alongside two non-LLM baselines (empty program, classical-CV heuristic). The strongest multimodal model by foreground IoU (GPT-5.5/`extra_high`) reaches mean foreground IoU 0.87, the best multimodal exact match remains 0.027, and the classical heuristic leads easy-tier exact match at 0.26 – a tier-dependent crossover that confirms ShapeCodeBench is not saturated and exposes distinct failure modes in perception versus structured code emission. Effort tier helps modestly (`max` > `high` for Claude on FG-IoU; `extra_high` > `medium` for GPT-5.5) but does not close either gap.

The rest of the paper is organized as follows. Section 2 places ShapeCodeBench against prior work. Section 3 describes the DSL, generator, renderer, and evaluator. Section 4 details the experimental setup and headline results. Section 5 analyzes failure modes, the heuristic-vs-LLM gap, and difficulty validity. Section 6 discusses limitations and future work.

2 Related Work

ShapeCodeBench sits at the intersection of three established lines of work: visual program induction, synthetic diagnostic benchmarks, and image-to-code evaluation of multimodal models.

Visual program induction and inverse graphics. Predicting executable programs from images has a long history. CSGNet [15] infers constructive solid geometry programs from 2D and 3D shapes, and is the direct conceptual ancestor of ShapeCodeBench. [12] learn to describe scenes with a DSL that supports loops and grouping, demonstrating that compositional program structure – and not only local shape identity – can be recovered from a single image. [9] and [8] extend program induction to perspective scenes and repeated 3D structure, while [4] studies parametric primitives with explicit function correlations, close in spirit to the integer-parameter DSL of ShapeCodeBench. LILO [5] synthesizes reusable program libraries across domains including graphics composition, providing a template for symbolic baselines. These systems prove that the image-to-program problem is well-defined; none of them are benchmark-first evaluations of modern multimodal models.

Benchmark design. CLEVR [6] showed how much scientific value a synthetic, carefully factorized benchmark can add when it is designed to reduce spurious shortcuts and expose reasoning failure modes. CLOSURE [1] extended this insight by probing systematic generalization. We adopt the same diagnostic posture: rather than building a large, noisy dataset, we expose the axes of variation explicitly and allow researchers to regenerate the dataset from a seed.

Renewable and dynamic evaluation. Renewability is becoming a first-class benchmark-design goal. LiveBench [17] adds and updates automatically scored questions to reduce test-set contamination and avoid the failure modes of human crowdsourcing or LLM-as-judge scoring on hard tasks. Image2Struct [14] similarly emphasizes fully automatic, renewable round-trip evaluation. ShapeCodeBench inherits this philosophy in a synthetic inverse-graphics setting: instead of downloading fresh natural data, it mints new controlled instances from fresh seeds and scores them through deterministic rendering.

Closest benchmark predecessors. TurtleBench [13] evaluates vision-language models on turtle-geometry programs and is the closest benchmark-level neighbor. It reports that strong models still struggle, which supports the general thesis that visual program reconstruction is hard. ShapeCodeBench differs along three axes: (1) the DSL is a tiny shape-primitive set rather than turtle paths, which removes path-planning reasoning and isolates perception-plus-emission; (2) the benchmark is explicitly renewable via fresh seeds; and (3) scoring is raster-based and deterministic.

Image2Struct [14] introduced round-trip structure extraction: image → structure → rendered image → similarity. Our scoring pipeline follows the same philosophy. Unlike Image2Struct, which spans noisy real-world web pages, LaTeX, and music, ShapeCodeBench stays inside a small controlled space so that reported failures can be cleanly attributed to perception or emission.

Broader image-to-code. pix2code [2], Design2Code [16], VCode [10], and Omni-I2C [19] demonstrate that image-to-code is now a mature benchmark family spanning GUI screenshots, SVG, and general graphics-to-code. These benchmarks mix many confounders: OCR, library conventions, rendering-engine variability, and external assets. ShapeCodeBench strips away these confounders by design, providing a sibling benchmark whose strengths are control and reproducibility rather than realism.

Verifiable feedback for training. Automatic execution feedback has also become an important training signal for code and reasoning models. CodeRL [7] and RLTF [11] use unit-test or execution feedback to improve code generation, while DeepSeek-R1 [3] and RLVE [18] illustrate the broader

role of verifiable rewards and procedurally generated environments in reinforcement learning for language models. These results motivate a future use of ShapeCodeBench as a verifiable training environment, but our present contribution is an evaluation benchmark: we do not train or fine-tune models on the task.

How to read the contribution. ShapeCodeBench is not the first benchmark to ask models to reconstruct programs from images. Its contribution is a *specific combination*: a deterministic render-based evaluator, explicit difficulty axes, a provider-agnostic adapter for cost-controlled runs, and a renewable frozen evaluation split with publicly verifiable instance hashes. We view it as complementary to TurtleBench and Image2Struct rather than a replacement.

3 Benchmark Design

ShapeCodeBench is specified by four coupled components: the DSL and its restricted parser, the scene generator, the deterministic renderer, and the render-based evaluator. We describe each in turn.

3.1 The ShapeCodeBench DSL

A ShapeCodeBench program is a sequence of top-level function calls, one per line. There are exactly four primitive functions:

```
filled_circle(cx=<int>, cy=<int>, radius=<int>)
circle(cx=<int>, cy=<int>, radius=<int>, stroke=<int>)
filled_square(cx=<int>, cy=<int>, size=<int>)
square(cx=<int>, cy=<int>, size=<int>, stroke=<int>)
```

The parser is implemented on top of Python's `ast` module but enforces a strict whitelist: only top-level expression statements; only calls to the four whitelisted names; only keyword arguments; only integer literals (including `+n`/`-n` unary forms). Imports, variables, loops, comprehensions, attribute access, starred arguments, duplicate keywords, and unexpected keywords are rejected with typed errors. Parameter ranges are validated: $\texttt{cx}, \texttt{cy} \in [0, 511]$; $\texttt{radius}, \texttt{size} \in [1, 512]$; $\texttt{stroke} \in [1, \texttt{radius}]$ for circles and $\texttt{stroke} \in [1, \lceil \texttt{size}/2 \rceil]$ for squares. Shapes may extend beyond the canvas and are clipped deterministically at render time.

The serializer is canonical: one call per line, fixed keyword order per primitive, normalized whitespace, no imports or boilerplate.

3.2 Renderer

The renderer uses Pillow's `ImageDraw` to produce a 512×512 8-bit grayscale image. Backgrounds are white (255) and shapes are black (0). `FilledCircle` uses `draw.ellipse(fill)`; `Circle` uses `draw.ellipse(outline, width=stroke)`; and the square counterparts use `draw.rectangle` with the same conventions. Program order is preserved in the render loop, but the binary palette makes scenes order-invariant: later shapes can add foreground pixels but cannot erase them. True order-sensitive evaluation is deferred to a future version.

3.3 Generator and Difficulty Tiers

The generator is seeded by a single integer and uses only `random.Random(seed)` for determinism. Scenes are produced by rejection-sampling candidate shapes until they satisfy tier-specific constraints

Tier	# shapes	Extent	Stroke	Clip prob.	Max bbox IoU	Overlap?
Easy	1–3	$[64, 160]$	$[2, 6]$	0.00	0.02	no
Medium	3–6	$[32, 128]$	$[2, 8]$	0.25	0.35	no
Hard	6–10	$[16, 128]$	$[1, 10]$	1.00	–	yes

Table 1: Difficulty tiers used to structure generation in ShapeCodeBench. "Extent" is the radius or size range; a missing Max bbox IoU indicates no constraint.

on (i) shape count, (ii) primitive extent (radius or size), (iii) stroke width, (iv) canvas clipping probability, and (v) the maximum allowed bounding-box IoU between the new shape and any existing shape. A tier may additionally require at least one pairwise bounding-box overlap.

Each generated sample is written to disk as a PNG together with a JSON metadata file containing the sample ID, split, difficulty, seed, canvas size, shape count, shape inventory, ground-truth program, and render configuration. Our frozen evaluation split eval_v1 uses contiguous seeds $0-49$ per tier, yielding 150 samples total; their SHA-256 hashes are published alongside the dataset.

3.4 Evaluator

Given a target image I_t and a predicted DSL program p, the evaluator attempts to parse p through the restricted parser, render the resulting scene into I_p, and compare rasters. We report five metrics.

- **Exact match** – 1 if $I_t = I_p$ pixel-exactly, else 0.
- **Pixel accuracy** – fraction of pixels equal between I_t and I_p.
- **Foreground IoU** – intersection-over-union of black pixels between I_t and I_p (convention: if both sets are empty, IoU is 1).
- **Parse success** – 1 if the parser accepts p, else 0.
- **Execution success** – 1 if rendering the parsed scene succeeds, else 0.

On parse or execution failure, all similarity metrics fall to 0 and the failure type is recorded for later analysis. We aggregate metrics over the full split and also report per-tier breakdowns.

4 Experiments

4.1 Protocol

We evaluate six systems on the frozen eval_v1 split: two non-LLM baselines and four reasoning-effort configurations across two frontier multimodal models. Exact invocation details are provided in Appendix A.

- **Empty-Program** – a floor baseline that always predicts an empty string; every sample fails parsing.
- **Heuristic-CV** – a classical-CV baseline that thresholds the image, labels connected components, classifies each component as a circle or square by bounding-box fill ratio, and as hollow or filled by morphological erosion. Stroke widths are estimated from the ratio of component area to estimated perimeter.

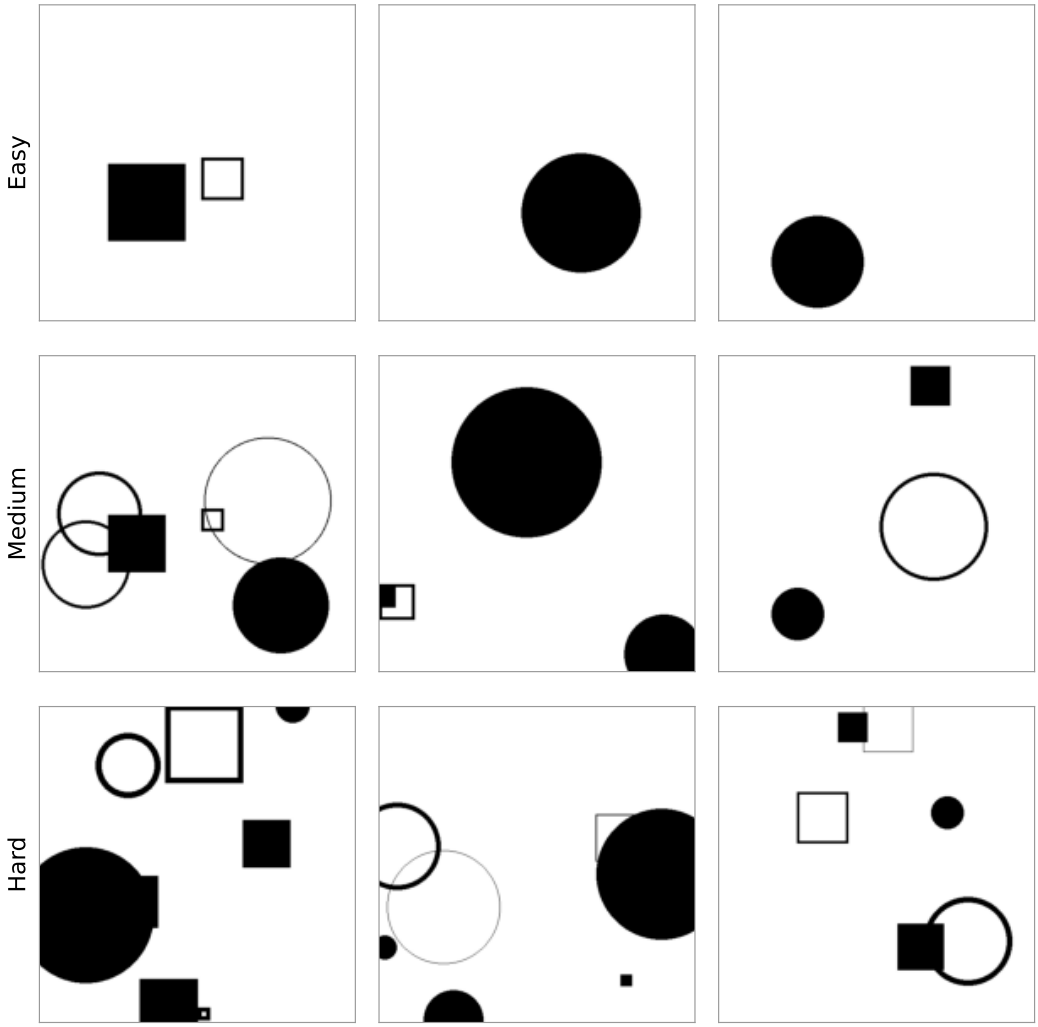

Figure 1: Representative samples from EVAL_V1 at each difficulty tier. The three tiers differ in shape count, extent, and overlap.

- **Claude Opus 4.7 (1M context)** at `high` and `max` effort.
- **GPT-5.5** at `medium` and `extra_high` reasoning effort.

All four LLM configurations share the same zero-shot prompt: a one-sentence system instruction ("Return only valid ShapeCodeBench DSL code. Do not include markdown fences, comments, or prose.") and a user block listing the four primitive signatures and formatting constraints. We do not use chain-of-thought prompting or few-shot examples. Raw model outputs are post-processed by a shared normalizer that prefers fenced code blocks anywhere in the response, falls back to primitive-signature line filtering, and ultimately surfaces the raw response so that parse errors are visible rather than masked.

4.2 Runner and artifacts

Each run writes a per-run directory under `data/runs/` containing `run_config.json`, `summary.json`, and per-sample JSON files with the request, raw and normalized predictions, usage, latency,

and the full evaluation result. All metrics in this paper are computed from these artifacts by `scripts/analyze.py`; figures are produced by `scripts/make_figures.py`. We use non-parametric bootstrap with 1000 resamples for 95% confidence intervals on per-difficulty means.

4.3 Headline results

Table 2 reports the aggregated metrics across all 150 samples for each system; 95% bootstrap CIs are shown in brackets. Figure 2 breaks the exact-match rate down by difficulty tier, and Figure 3 shows the same decomposition for all four scored metrics.

Model	Exact	PixAcc	FG-IoU	Parse
Heuristic-CV	0.087 [0.047, 0.133]	0.881 [0.862, 0.898]	0.583 [0.535, 0.628]	1.000 [1.000, 1.000]
gpt-5.5 (medium)	0.027 [0.007, 0.053]	0.963 [0.937, 0.984]	0.850 [0.813, 0.884]	0.973 [0.947, 0.993]
gpt-5.5 (extra_high)	0.020 [0.000, 0.040]	0.983 [0.969, 0.991]	0.865 [0.836, 0.893]	0.993 [0.980, 1.000]
Empty-Program	0.000 [0.000, 0.000]	0.000 [0.000, 0.000]	0.000 [0.000, 0.000]	0.000 [0.000, 0.000]
claude-opus-4-7[1m] (high)	0.000 [0.000, 0.000]	0.900 [0.873, 0.919]	0.439 [0.403, 0.474]	0.980 [0.960, 1.000]
claude-opus-4-7[1m] (max)	0.000 [0.000, 0.000]	0.919 [0.912, 0.927]	0.461 [0.427, 0.495]	1.000 [1.000, 1.000]

Table 2: Main results on EVAL_V1. "Exact" is pixel-exact raster match; "PixAcc" is mean pixel accuracy; "FG-IoU" is mean foreground IoU; "Parse" is parse success rate. Brackets give 95% bootstrap CIs.

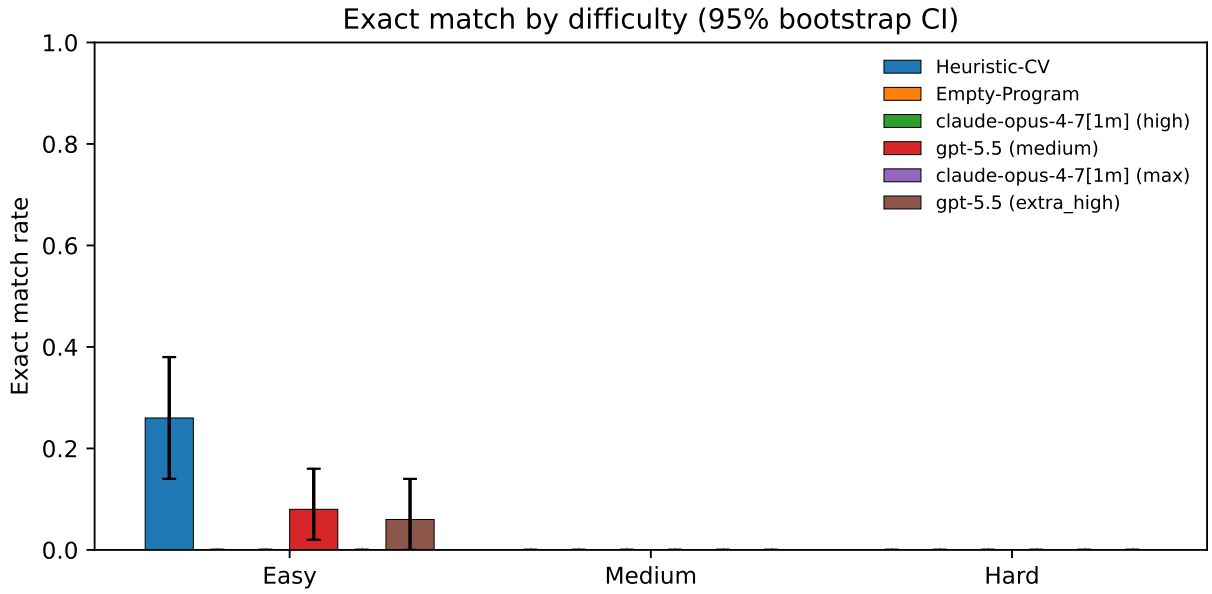

Figure 2: Exact-match rate by difficulty tier. Error bars are 95% bootstrap CIs over per-sample indicators.

The key qualitative pattern – detailed in Section 5 – is that exact-match collapses on the hard tier for every system, while foreground IoU degrades more gracefully; the heuristic baseline is surprisingly competitive on easy scenes and is outclassed on hard scenes, where LLMs can enumerate and place multiple overlapping shapes that classical connected components cannot individuate.

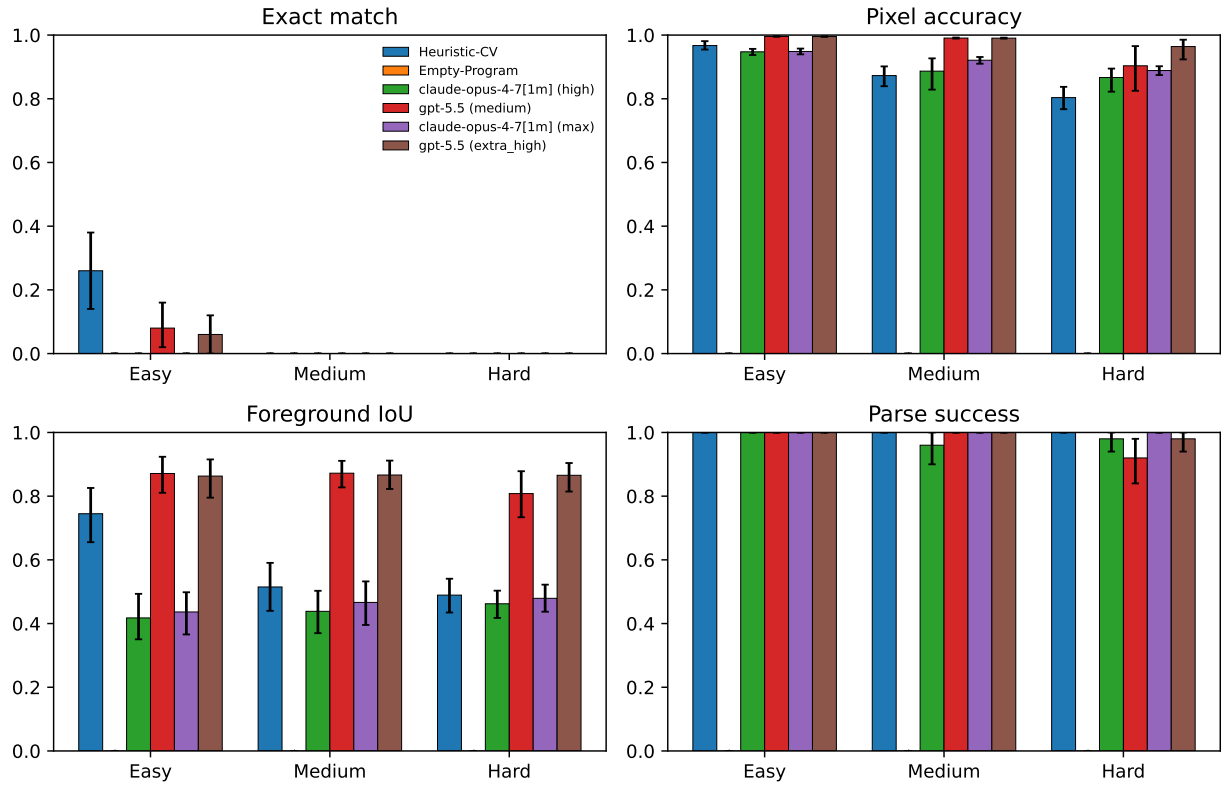

Figure 3: All four scored metrics by difficulty tier. The four panels are exact-match, pixel accuracy, foreground IoU, and parse success.

5 Analysis

5.1 Error taxonomy

Figure 4 reports the distribution of evaluator error types per model. Three patterns are worth flagging.

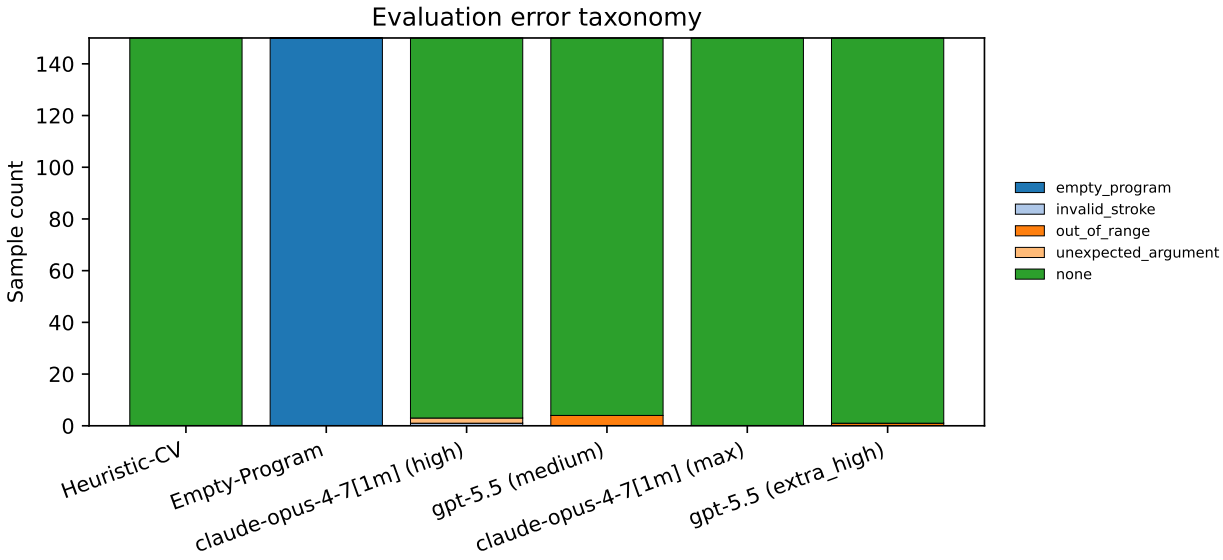

Figure 4: Evaluator error taxonomy per model. The "none" bar counts samples whose predictions parsed and rendered without error, including samples that did not pixel-match the target. Remaining categories surface structured parsing or validation failures.

First, `Empty-Program` concentrates all 150 samples under the `empty_program` parse error, because an empty DSL program is a parse failure by construction. This is the intended floor and not a pathology.

Second, the LLM runs have non-zero but small parse-failure counts, dominated by `out_of_range` (predicted coordinates or extents outside the valid $[0, 511]$ and $[1, 512]$ ranges) and `invalid_stroke` (stroke widths exceeding the primitive's documented limit). These are cases where the model understood the task format but violated structural constraints – the kind of failure mode that shows models do not internalize the DSL's range constraints from a short prompt alone.

Third, the `Heuristic-CV` baseline has *zero* parse failures because it only emits programs it can itself construct. Its errors manifest as low foreground IoU rather than parse failures.

5.2 Qualitative wins and losses

Figure 5 shows representative wins (top rows) and losses (bottom rows) for the best exact-match multimodal configuration: each row shows the target image, the prediction rendered through the same renderer, and the XOR diff of foreground masks. The wins are dominated by the easy tier, where two or three non-overlapping shapes can be located and parameterized precisely. Losses tend to come from medium and hard tiers and split into three recurring categories: (i) correct shape inventory but off-by-a-few-pixel parameter estimates, (ii) missed occluded shapes in high-overlap hard scenes, and (iii) misclassification of hollow vs. filled when stroke widths are thin.

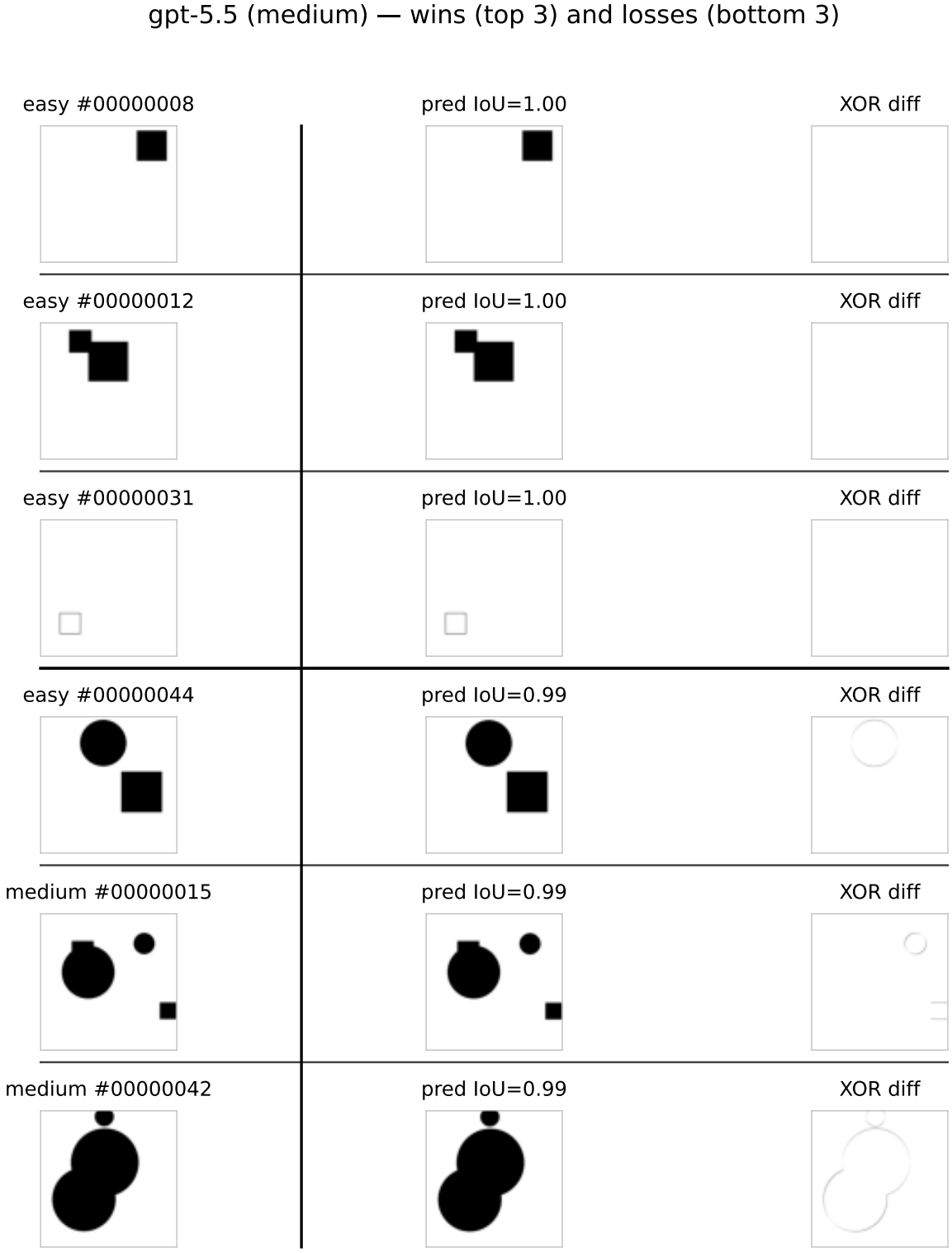

Figure 5: Wins (top) and losses (bottom) for the best exact-match multimodal configuration on EVAL_V1 (GPT-5.5 at `medium` reasoning effort, selected automatically by exact-match rate). Each row shows the target, the re-rendered prediction, and a foreground-XOR diff. Losses cluster on overlapping hard scenes and stroke-width confusions.

5.3 Difficulty validity

A renewable benchmark is useful only if its difficulty tiers track real performance gradients. Figure 2 shows that exact-match rate falls monotonically from easy to hard for every system, including the heuristic baseline. Foreground IoU tracks the same ordering but with shallower slopes, as expected: pixel-level similarity degrades more gracefully than the all-or-nothing exact-match metric. This monotonic structure supports the claim that EVAL_V1's difficulty axes are not arbitrary.

5.4 Heuristic vs. LLM gap

The heuristic baseline anchors what a purely bottom-up computer-vision pipeline with no DSL-level reasoning can achieve. Its story splits by tier. On the easy tier it is surprisingly competitive on *exact-match*, because easy scenes are separated and unclipped – connected components match shapes directly, the area/perimeter stroke estimator is close enough, and the hollow-vs-filled test on eroded masks rarely errs. Multimodal models, by contrast, almost always miss exact-match on easy scenes by a few pixels of parameter error: they recover the *right shapes* but not the *right parameter values*.

On the medium and hard tiers the picture flips. Foreground IoU for the heuristic deteriorates sharply because overlapping or clipped shapes merge into a single connected component and the pipeline can no longer individuate them. Multimodal models retain most of the spatial structure (foreground IoU stays roughly flat across tiers) but still do not pixel-match – their programs contain the right number of shapes in roughly the right positions, but they cannot land parameters precisely under occlusion.

Taken together, the heuristic is a better *exact-match* baseline on easy scenes and a worse *IoU* baseline on harder ones. This decomposition is precisely the kind of diagnostic signal SHAPECODEBENCH is designed to produce: difficulty is not a single-axis phenomenon, and different systems fail in different places. The sharp easy-tier exact-match comparison also argues that closing that gap – having a model emit *both* the right shape list *and* the right parameter values on clean scenes – is a natural first-order target for future work.

6 Limitations and Future Work

SHAPECODEBENCH makes deliberate scoping choices in its v1 incarnation, each of which is a direction for future work.

Monochrome palette. V1 is black-on-white. This collapses draw-order sensitivity: later shapes can paint foreground pixels but cannot erase or overwrite earlier ones. A richer palette (multiple colors or an explicit `clear` primitive) would make draw order first-class and enable sharper compositionality tests.

Four primitives. The DSL currently covers only filled and hollow circles and squares. Extending to rectangles, lines, polygons, or parametric curves would stress different kinds of visual reasoning and likely move saturation further out.

Zero-shot only. We evaluate without chain-of-thought prompting or few-shot examples. These are natural knobs to explore and may change the ordering of models, especially for the reasoning-heavy hard tier.

Model-inference variability. The frozen evaluation images, parser, renderer, and scorer are deterministic: regenerating EVAL_V1 from the published seeds should reproduce the same target PNGs and metric computation. The remaining variability is in model inference. Repeating the same image-and-prompt request to a hosted multimodal model may produce a different predicted program, so we cannot guarantee bit-exact reproduction of the reported model scores. Each run's `run_config.json` records the model configuration and invocation settings needed to reproduce the experimental setup.

No human baseline. We do not report human performance. An informal human baseline on a small sample of scenes would make the "how hard is this really?" claim concrete, and we plan to add it in a subsequent revision.

Model coverage. We evaluate two frontier multimodal models – CLAUDE OPUS 4.7 (1M context) and GPT-5.5 – across two reasoning-effort tiers each. Other frontier multimodal systems are not covered here but can be added by implementing a new adapter against the same `ModelAdapter` Protocol.

Contamination resistance. The current eval split's seeds are public. We rely on the *renewability* of the benchmark rather than on secrecy: to evaluate under a contamination-free setting, regenerate EVAL_VN from fresh seeds using the published generator and manifest. This is intentionally cheap: the entire 150-sample split regenerates in under a second.

Training signal, not a training pipeline. The current evaluator is an offline benchmark, not a differentiable pretraining loss. Predictions are discrete DSL text, the parser is symbolic, the renderer is Pillow-based, and the reported metrics are computed after hard rasterization. Future training use would therefore require a separate setup, such as supervised fine-tuning on generated image/program pairs, reinforcement learning or rejection sampling with render-based rewards, a learned reward or critic model, or a differentiable renderer variant. Any such use should train on generated training seeds and reserve EVAL_V1 or fresh held-out splits for clean evaluation.

Reproducibility. We publish the benchmark code, the frozen EVAL_V1 dataset with SHA-256 hashes per image, per-run artifacts, analysis scripts, and paper sources. `docs/REPRODUCIBILITY.md` walks through the end-to-end workflow from a clean checkout.

Acknowledgments

We thank the authors of the prior benchmarks cited throughout this paper for laying the groundwork that SHAPECODEBENCH builds on, and the maintainers of `Pillow`, `numpy`, and `scipy` for tools that made the evaluator and heuristic baseline straightforward to implement.

References

[1] Dzmitry Bahdanau, Shikhar Murty, Michael Noukhovitch, Thien Huu Nguyen, Harm de Vries, and Aaron Courville. Closure: Assessing systematic generalization of clevr models. *arXiv preprint arXiv:1912.05783*, 2019.

[2] Tony Beltramelli. pix2code: Generating code from a graphical user interface screenshot. *arXiv preprint arXiv:1705.07962*, 2017.

[3] DeepSeek-AI, Daya Guo, et al. Deepseek-r1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[4] Xuguang Duan, Xin Wang, Pengchuan Zhao, Guangyao Shen, and Wenwu Zhu. Parametric visual program induction with function modularization. In *International Conference on Machine Learning (ICML)*, 2022.

[5] Gabriel Grand, Lionel Wong, Matthew Bowers, Theo X. Olausson, Muxin Liu, Joshua B. Tenenbaum, and Jacob Andreas. Lilo: Learning interpretable libraries by compressing and documenting code. *arXiv preprint arXiv:2310.19791*, 2024.

[6] Justin Johnson, Bharath Hariharan, Laurens van der Maaten, Li Fei-Fei, C. Lawrence Zitnick, and Ross Girshick. Clevr: A diagnostic dataset for compositional language and elementary visual reasoning. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.

[7] Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C. H. Hoi. Coderl: Mastering code generation through pretrained models and deep reinforcement learning. *arXiv preprint arXiv:2207.01780*, 2022.

[8] Yikai Li, Jiayuan Mao, Xiuming Zhang, William T. Freeman, Joshua B. Tenenbaum, and Jiajun Wu. Multi-plane program induction with 3d box priors. *arXiv preprint arXiv:2011.10007*, 2020.

[9] Yikai Li, Jiayuan Mao, Xiuming Zhang, William T. Freeman, Joshua B. Tenenbaum, and Jiajun Wu. Perspective plane program induction from a single image. *arXiv preprint arXiv:2006.14708*, 2020.

[10] Lin et al. Vcode: a multimodal coding benchmark with svg as symbolic visual representation. *arXiv preprint arXiv:2511.02778*, 2025.

[11] Jiate Liu, Yiqin Zhu, Kaiwen Xiao, Qiang Fu, Xiao Han, Wei Yang, and Deheng Ye. Rltf: Reinforcement learning from unit test feedback. *arXiv preprint arXiv:2307.04349*, 2023.

[12] Yunchao Liu, Jiajun Wu, Zhijian Wu, Daniel Ritchie, William T. Freeman, and Joshua B. Tenenbaum. Learning to describe scenes with programs. In *International Conference on Learning Representations (ICLR)*, 2019.

[13] Sina Rismanchian et al. Turtlebench: A visual programming benchmark in turtle geometry. *arXiv preprint arXiv:2411.00264*, 2025.

[14] Jonathan Roberts et al. Image2struct: Benchmarking structure extraction for vision-language models. In *NeurIPS Datasets and Benchmarks*, 2024.

[15] Gopal Sharma, Rishabh Goyal, Difan Liu, Evangelos Kalogerakis, and Subhransu Maji. Csgnet: Neural shape parser for constructive solid geometry. *arXiv preprint arXiv:1712.08290*, 2018.

[16] Chenglei Si, Yanzhe Zhang, Zhengyuan Yang, Ruibo Liu, and Diyi Yang. Design2code: How far are we from automating front-end engineering? In *arXiv*, 2024.

[17] Colin White, Samuel Dooley, Manley Roberts, Arka Pal, Ben Feuer, Siddhartha Jain, Ravid Shwartz-Ziv, Neel Jain, Khalid Saifullah, Sreemanti Dey, Shubh-Agrawal, Sandeep Singh Sandha, Siddartha Naidu, Chinmay Hegde, Yann LeCun, Tom Goldstein, Willie Neiswanger, and Micah Goldblum. Livebench: A challenging, contamination-limited LLM benchmark. *arXiv preprint arXiv:2406.19314*, 2024.

[18] Zhiyuan Zeng, Hamish Ivison, Yiping Wang, Lifan Yuan, Shuyue Stella Li, Zhuorui Ye, Siting Li, Jacqueline He, Runlong Zhou, Tong Chen, Chenyang Zhao, Yulia Tsvetkov, Simon Shaolei Du, Natasha Jaques, Hao Peng, Pang Wei Koh, and Hannaneh Hajishirzi. Rlve: Scaling up reinforcement learning for language models with adaptive verifiable environments. *arXiv preprint arXiv:2511.07317*, 2025.

[19] Zhou et al. Omni-i2c: A holistic benchmark for high-fidelity image-to-code generation. *arXiv preprint arXiv:2603.17508*, 2026.

A Reproducibility Details

The main paper reports model identifiers and reasoning-effort settings; this appendix records the operational path used for the reported sweeps. The full step-by-step workflow is maintained in `docs/REPRODUCIBILITY.md`, and each run also writes its exact configuration to `data/runs/<run_id>/run_config.json`.

OpenAI Codex invocation. GPT-5.5 runs used the OpenAI Codex CLI, authenticated through the author's ChatGPT subscription. The adapter invokes this command shape:

```
codex exec --skip-git-repo-check --ephemeral -s read-only \
  -m gpt-5.5 -i <image_path> -o <last_message_path> \
  -C <temporary_workdir> --color never \
  -c reasoning_effort=<medium|extra_high>
```

The image is attached once per request, and `-o` captures the final message for normalization. The paper sweeps used two retries with exponential backoff and per-sample timeouts of 1800 seconds for `medium` and 2400 seconds for `extra_high`.

Claude invocation. Claude Opus 4.7 (1M context) runs used the Claude Code CLI, authenticated through the author's Claude subscription. The adapter invokes this command shape:

```
claude --print --add-dir <image_parent> \
  --model claude-opus-4-7[1m] --effort <high|max> \
  --output-format text --no-session-persistence
```

The target image is referenced in the prompt as `@<image_path>`, and `-add-dir` grants read access to the sample directory. The paper sweeps used two retries with exponential backoff and per-sample timeouts of 1800 seconds for `high` and 2400 seconds for `max`.

Artifacts. Both multimodal paths receive the same zero-shot prompt and are normalized by the same prediction normalizer before parsing and rendering. Run artifacts include the raw response, normalized response, latency, adapter metadata, full evaluation result, and aggregate summaries. The non-LLM baselines use the same runner and artifact schema but do not call an external model-serving interface.